

科学计算 子系统

统计内核 · 科研 I/O · 绘图引擎 · 可复现性

src/core 下有四个纯 TypeScript、少 DOM、Node 环境可测的科学计算子系统：统计内核（stats）、科研二进制 I/O（io）、出版级绘图引擎（plot）与可复现性内核（repro）。它们被区块系统、Studio API 与工作台界面共同消费，是产品的“科学”底座——统计与绘图保证结果的专业性，I/O 保证数据入口的开放性，可复现性保证结论可以被别人原样重放。

四个子系统

stats

六模块统计内核
t 检验 · 方差分析

io

五种科研格式
按魔数分派

plot

纯 TS 绘图
SVG 与 PDF 导出

repro

种子 · 清单
DAG 转 Python

目 录

- 一、统计内核 (src/core/stats) 03
- 二、科研二进制 I/O (src/core/io) 04
 - 2.1 统一调度与格式清单 04
 - 2.2 辅助工具 05
- 三、出版级绘图引擎 (src/core/plot) 06
- 四、可复现性内核 (src/core/repro) 07
- 五、支撑设施 08

阅读提示：四个子系统全部为纯函数实现，这也决定了它们的消费方式—— 流程模式的统计与绘图区块、积木与代码模式的 studio.summary 与 studio.plot， 底层都指向这里的同一份实现，不存在"两种算法"。

一 统计内核 (src/core/stats)

统计内核消费普通数值数组、DataTable 或 Dataset，返回结构化结果，全部为纯函数实现。公共出口按模块划分：

模块	能力
descriptive	均值、方差、标准差、中位数、分位数、摘要统计、均值置信区间（t 分布临界值经 studentTInv 求得）
special	特殊函数：不完全伽马与贝塔函数、正态与 t 分布等累积分布及其逆函数（供检验与功效计算复用）
tests	单样本 t 检验、双样本 t 检验（Welch 校正自由度）、配对 t 检验、单因素方差分析、Mann-Whitney U 检验、卡方独立性检验，返回含统计量、p 值与自由度的结构化结果
effect	Cohen's d 效应量、Pearson 与 Spearman 相关
correction	Bonferroni 与 Benjamini-Hochberg 多重比较校正，p 值先经清洗（非有限值剔除）再校正
power	双样本 t 检验的功效分析

边界输入一律显式抛错而不是静默给出错误数字：Welch 自由度除零、方差分析组数不足、配对检验长度不等都会得到结构化错误。统计能力向用户的呈现方式有两种：流程模式的 11 个 stats 区块（t 检验、方差分析、非参数检验、卡方检验、相关、效应量、校正等），以及积木与代码模式 studio 接口中的 summary 等调用，底层都指向同一实现。配套单元测试覆盖描述统计、特殊函数、各类检验、效应量、校正与功效六个测试文件。

特殊函数模块（special）是整个内核的地基：t 检验的 p 值、置信区间的临界值、功效分析全都依赖 不完全伽马与贝塔函数的实现。它不直接面向用户，却决定了每一个统计数字的正确性——因此它有独立的单元测试，与教科书数值逐项对照。

科研二进制 I/O (src/core/io)

2.1 统一调度与格式清单

科研数据常以领域二进制格式存储。io 子系统提供单一入口 `loadScientificData`：给定拖入或打开的 `File`，先读文件头并按魔数（或扩展名）识别格式，再路由到对应加载器，把每个变量、数据集或 HDU 转成统一的 `RawVariable` 列表，调用方负责将其转为项目数据文件。

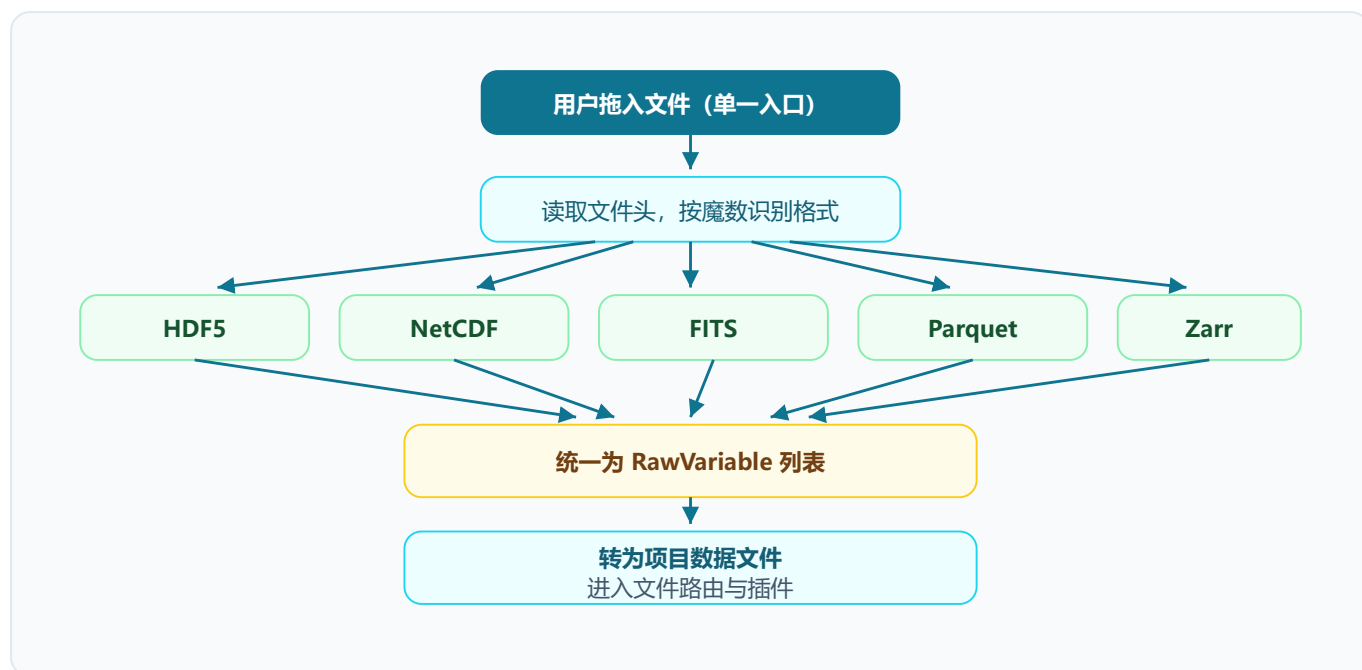


图 1 科研 I/O 统一调度：五类格式各自解析，汇入同一份 `RawVariable` 列表

格式	典型领域	依赖库
HDF5	通用科学分层数据	h5wasm
NetCDF	地学、气象	netcdfjs
FITS	天文图像与表	fitsjs
Parquet	列式表格	parquet-wasm (经 apache-arrow 合并全部记录批次)
Zarr	分块 N 维数组 (本地目录或远程存储)	zarrita

2.2 辅助工具

辅助工具 (io/types) 包括整型到 Float64 的安全转换 (int64 与 uint64 数据集不再抛错)、名称净化、DataTable 转 CSV 与数据集构造; NetCDF 加载器与辅助工具各有专门单元测试。这些细节解决的是真实数据里的"脏"问题: 科学数据里 64 位整数很常见, 而 JavaScript 的 Number 无法安全表示全部范围, 统一转成 Float64 并在转换边界做检查, 比在每个消费点各自处理要可靠得多。

☰ 出版级绘图引擎 (src/core/plot)

区别于插件里的画布渲染, plot 子系统是一套纯 TS 的矢量绘图引擎, 目标是把 DataTable 转成出版级 SVG 并导出:

模块	能力
types	PlotSpec、PlotSeries、比例类型与 SVG 载荷定义
scale	线性、对数、时间刻度, niceTicks 优雅刻度值, 刻度格式化; 退化定义域 (单点、零跨度) 有明确处理
charts	DataTable 转折线、散点、直方图、柱状四类图型
svg	renderSVG 矢量渲染, 属性与文本分别转义
export	导出 SVG 文件与 PDF, 下载文本

由于整条链路 (数据、刻度、渲染、导出) 都是纯函数, 绘图引擎有专门的单元测试 (tests/plot), 也是可复现性内核 DAG 转 Python 导出策略的天然搭配。流程模式的绘图区块 (plot.line、plot.scatter、plot.histogram、plot.bar) 即由这套引擎渲染, 与插件画布渲染互为补充: 前者产出可导出的矢量文件, 后者产出可交互的实时画面。

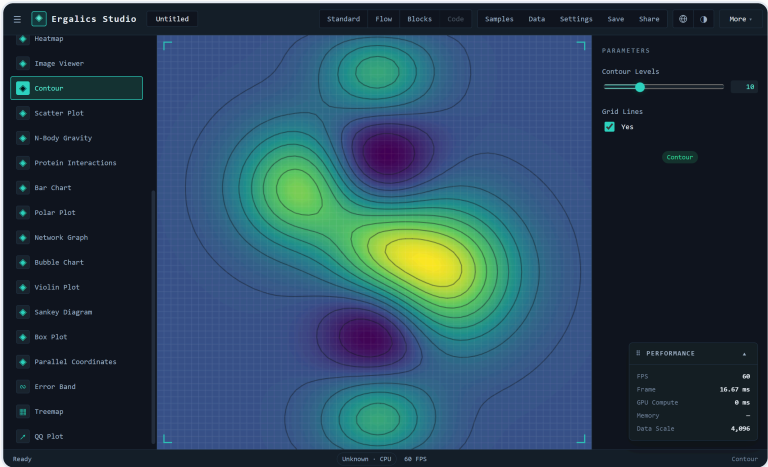


图 2 等值线插件渲染的双高斯标量场 (插件画布渲染路径的代表)

四 可复现性内核 (src/core/repro)

科研计算的可复现性要求“同样的输入与参数得到同样的输出”。repro 内核提供四块能力：

1. **带种子的伪随机数**：mulberry32 算法，同种子产生完全一致的序列；全局 setSeed 与 currentSeed 支持会话级设定。
2. **稳定哈希**：hashString 输出稳定的八位十六进制，用于输入与图的指纹。
3. **运行清单**：createManifest 记录种子、工作室版本、每个输入文件的哈希、区块图（经拓扑排序后计算哈希）与输出；manifestToText 生成可归档的文本清单。
4. **DAG 转 Python**：把流程图导出为可独立重跑的 Python 脚本，与 topoSort 依赖排序配合，导出脚本内含与运行清单一致的种子设定。

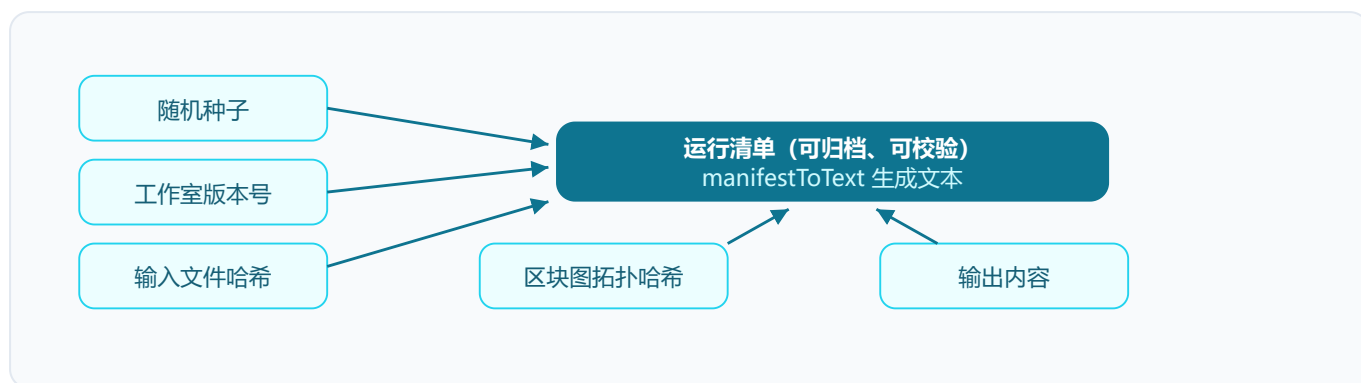


图 3 运行清单的五项构成：种子、版本、输入哈希、图哈希与输出

运行清单的意义在于把“复现”从口头承诺变成可校验的文件：拿到清单的人可以核对每个输入的哈希、重设同一种子、重跑同一张图，逐位比对自己的输出。区块图哈希取的是拓扑排序后的结构指纹，节点位置这类与语义无关的信息不参与计算，画布上挪动节点不会使清单失效。

五 支撑设施

- **运行日志导出**：src/core/logger.ts 聚合会话日志，配合 download.ts 支持从工作台导出日志文件，便于问题反馈与回归定位。
- **插件缓存与示例资产**：pluginCache.ts 与 exampleAssets.ts、examples.ts 支撑内置示例数据与示例项目（examples/projects 下 11 个流程管线、examples/code 下 9 个 Python 示例）的发现与加载。
- **viewport2d 与 dataFiles**：2D 视口抽象（快照式访问，插件卸载安全）与项目数据文件注册（setProjectFiles 与 resolveDataFile），使项目自有文件、内置示例与代码模式 _FILES 共享同一套加载逻辑。

四个子系统的共同纪律：纯函数、少 DOM、Node 环境可测、边界显式抛错。这不是风格偏好，而是产品"科学"属性的工程保障——统计数字、数据解析与绘图输出一旦依赖界面状态或浏览器环境，就既无法单元测试，也无法在可复现性意义上被信任。

这四个子系统共同构成产品的"科学"底座：统计与绘图保证结果的专业性，I/O 保证数据入口的开放性，可复现性保证结论可以被别人原样重放。它们与前三篇描述的插件系统、四模式编辑器与 GPU 计算层拼合，才是完整意义上的"浏览器中的科学计算工作站"。